ASSIGNMENT 2 – CO2

Q7: DIAGNOSE, REDESIGN AND VALIDATE

Selected case: Railway e-ticket text construction (Q1)

Technical design-challenge report | Student-identifying details intentionally omitted

A. Diagnose the bottleneck (2 marks)

The selected limitation is the original Q1 approach that constructs each ticket with repeated String concatenation inside a loop. Java String objects are immutable. Each `result += field` creates a new String containing the previous accumulated characters and the next field. The growing prefix is copied repeatedly, and obsolete intermediate strings become eligible for garbage collection. The costly operation is therefore repeated copying during ticket construction, not the later storage or checksum of the completed ticket.

The earlier Q1 evidence recorded a mean of 40.695 ms for 10,000 tickets with 50 fields using String, compared with 5.025 ms using StringBuilder and 5.515 ms using StringBuffer. For 1,000 tickets with 200 fields, the earlier means were 34.550 ms for String and 2.808 ms for StringBuilder. The Q1 source-level estimate for intermediate String objects was 490,000 in the high-load case and 199,000 in the wide-ticket case. These values identify repeated concatenation as the bottleneck.

B. Define the design objective (2 marks)

Replace repeated immutable concatenation with a mutable per-ticket buffer while preserving the exact ticket text. Measurable objectives: (1) output equality must pass for every tested workload; (2) the estimated intermediate String count caused by per-field concatenation should fall from tickets × (fields – 1) to zero; and (3) average generation time should decrease on the same high-load and wide-ticket workloads. The design keeps one final String per ticket for downstream use.

C. Design the revised solution (4 marks)

The redesigned implementation creates a local StringBuilder for each ticket. It first sums the lengths of all pre-built fields and uses that value as the initial capacity, then appends each field and calls toString() once. Pre-sizing avoids repeated internal buffer growth in addition to avoiding intermediate immutable String concatenations. The builder is local to one build operation, so no synchronization is required and concurrent booking threads do not share mutable builder state.

Pseudocode

```
buildTicket(fields):
capacity = sum(length(field) for each field)
builder = new StringBuilder(capacity)
for each field in fields: builder.append(field)
return builder.toString()
```

Conceptual class flow: TicketRedesign → Builder strategy → {Original String concatenation | Redesigned pre-sized StringBuilder} → final ticket String → checksum/output verification. Both strategies receive the same pre-built field array and ticket count.

The redesign changes only ticket assembly. It does not alter field content, separators, ticket count, output retention or measurement boundaries. Additional memory is required for the mutable builder's character storage while a ticket is being built. Its capacity is proportional to the final output length. The final String is still allocated. The trade-off is one builder allocation per ticket and a short preliminary pass to calculate capacity; this is offset by eliminating repeated prefix copying and many temporary Strings.

D. Predict before implementing (1 mark)

Prediction: for a workload with many fields, the StringBuilder redesign should be substantially faster and should reduce the source-level estimate of intermediate String objects to zero. The benefit should be small for one short ticket but large for thousands of tickets or hundreds of fields. The time complexity for bounded-size fields changes from $O(n^2)$ character copying for repeated concatenation to $O(L)$ for the builder, where L is the total number of output characters ($O(n)$ when field lengths are bounded).

E. Implement and validate the redesign (2 marks)

TicketRedesign.java implements both approaches and runs them on identical pre-built input fields. It performs three warm-up rounds, five measured repetitions, alternates measurement order, times only the ticket-generation loop using System.nanoTime(), verifies output equivalence, consumes results through a checksum and writes each measurement to task7_results.csv. The two workloads match the Q1 high-load and wide-ticket conditions.

Workload	Original String	Redesigned StringBuilder	Speed-up	Intermediate Strings (old → new)
10,000 × 50 fields	42.186 ms	5.135 ms	8.22×	490,000 → 0
1,000 × 200 fields	39.078 ms	1.894 ms	20.63×	199,000 → 0

In the prepared validation run, both workloads passed output equivalence. The measured averages above are generated by the included program in the preparation environment (OpenJDK 21.0.12.1); your computer may produce different absolute timings. The raw CSV stores five repetitions for each approach and workload. The earlier Q1 report values are retained as historical baseline evidence; the new Task 7 run is the direct A/B validation using the same workload sizes and input-generation approach.

F. Defend the final design (2 marks)

The measured result supports the prediction: the redesigned StringBuilder implementation reduced average construction time in both workloads and reduced the estimated intermediate String count from 490,000 to zero for the high-load case and from 199,000 to zero for the wide-ticket case. The final ticket String is still created once per ticket. The measured speed-up is workload- and environment-dependent and should be rechecked on the submission computer.

The redesign's new costs are allocation of a mutable builder for each ticket and a pass to calculate exact capacity. Pre-sizing may not be worthwhile for a single tiny ticket, and if a reliable capacity estimate is unavailable, a normal StringBuilder with default capacity remains a sound linear-time choice. The original String approach is acceptable for a few fixed concatenations outside loops where simplicity matters more than allocation cost. For a per-ticket builder confined to one booking thread, StringBuilder is preferable to StringBuffer because cross-thread synchronization is unnecessary. A shared mutable booking log is a separate concurrency problem and should use appropriate synchronization or a dedicated logging mechanism.

Conclusion

The diagnosed bottleneck was repeated immutable String concatenation in Q1. A per-ticket, pre-sized StringBuilder directly addresses repeated prefix copying while preserving output. The included experiment validates output equivalence and compares the original and redesigned implementations over two larger workloads with repeated measurements. The redesign is recommended for high-volume ticket construction; simple concatenation remains reasonable for very small fixed expressions.

Reproduction

From the Task07 folder run `javac TicketRedesign.java` followed by `java TicketRedesign`. The program creates task7_results.csv and prints average timings and output-equivalence checks. Capture a terminal screenshot from your own execution and include the regenerated CSV in your submission.